RL-Course 2024/25: Final Project Report

Reinforce the Puck: Jonathan Schwab, Tom Freudenmann

February 24, 2025

1 Introduction

In this project, we implemented and evaluated three reinforcement learning agents for a hockey environment: Twin Delayed Deep Deterministic Policy Gradient (TD3), Soft Actor Critic (SAC), and a hybrid model combining both. The goal was to develop these agents from scratch and optimize their performance with tailored modifications, including improved replay buffers, reward shaping, exploration noise strategie and asymeetric self-play.
The hockey environment requires precise control and strategic decision-making in an continuous state and action space. We designed a PyTorch-based framework supporting multiple environments, enabling us to train, finetune and evaluate our agents on different tasks.
Our implementation follows the original research papers with modifications to enhance training stability and efficiency. We introduced Balanced Experience Replay (BER) and Prioritized Experience Replay (PER) to improve sample efficiency and used pink noise for better exploration.
To further enhance performance, we developed a hybrid model that selects between TD3 and SAC using a Double Deep Q-Network (DDQN) gate, dynamically choosing the best agent per state.
We first evaluated our models on standard benchmarks (Pendulum, Lunar Lander, and HalfCheetah) before training them in the hockey environment. This report details our methodology, experimental setup, and results.

2 Method

2.1 Framework

We implemented a PyTorch-based framework that abstracts agents and environments, ensuring they are interchangeable and easily configurable for training. In a configuration file, the environment[1], the agent type, its training parameters, hyperparameters, and opponents can be specified.
The framework also enables (asynmmetric) self-play of the agents, where the trained agent is periodically replaced by one of its opponents at a specified frequency. The rule-based agents of the hockey environment can also be used as opponents but are not considered as training candidates.
The leaderboard of our framework ranks agents dynamically based on their performance using the Placket-Luce rating system [8]. There, each agent has a skill rating average μ and variance σ, which are updated after every match. Games are simulated, and winners gain rating points while losers may lose them. Agents are paired based on a probability distribution over their skill levels, ensuring fair matchups. To ensure a diverse tournament, we first sample 4 agents and assign them probabilities based on the softmax of their σ. Based on them, the final 2 agents are sampled. Agent names, types, ratings, and average scores are displayed on the leaderboard.

[1] Our framework supports the OpenAI Gymnasium environment and the Hockey environment.

2.2 Twin Delayed Deep Deterministic Policy Gradient[2]

Our TD3 algorithm is a from scratch implementation of the original paper [2]. TD3 is an off-policy actor-critic algorithm for continuous action and state spaces. This combination is very vulnerable to overestimation bias and error accumulation in Temporal Difference (TD)-learning [2]. The overestimation bias is related to discrete Q-learning, where the value estimate is updated with the following greedy target:

$$y = r + \gamma \max_{a'} Q(s', a') \leq \mathbb{E}_{\epsilon} \left[max_{a'}(Q(s', a') + \epsilon) \right] \tag{1}$$

Where r is the reward, γ the discount factor, s' the next state, and a' the next action. Thus, the result is generally larger than the true maximum for the estimate with the error ϵ [12]. In actor-critic environments, the policy is updated with a deterministic policy gradient, which leads to suboptimal updates and diverging behavior in the learning process [2].
Therefore, TD3 introduces clipped double Q-Learning that is based on DDQN. DDQN optimizes two actors with two critics, which use the same replay buffer and the opposite critic in the learning process [14], inducing an overestimation of some states. In contrast, TD3 uses a single actor π_ϕ which is optimized with respect to the minimum of two critics $Q_{\theta_1}, Q_{\theta_2}$:

$$y = r + \gamma \min_{i=1,2} Q_{\theta'_i}(s', \pi_\phi(s')) \tag{2}$$

As a result, the computational costs are reduced because a single actor is used. But instead, an underestimation bias is induced that is preferable to overestimation since it does not propagate through the policy updates [2].
Another challenge are the high variance estimates, causing a noisy policy gradient. Thus, the Bellman equation is never exactly satisfied, resulting in a small residual TD-error δ_t. This error propagates through the update step, such that the expected return depends on the expected discounted difference between reward and TD-error [2]:

$$Q_\theta(s_t, a_t) = \mathbb{E}_{s_i \sim p_\pi, a_i \sim \pi} \left[\sum_{i=t}^{T} \gamma^{i-t}(r_i - \delta_i) \right] \tag{3}$$

TD3 addresses this problem by using slowly changing target networks for actors and critics and introducing policy smoothing regularization. Firstly, target networks are used to reduce the error over multiple updates by providing a stable learning objective and reducing the variance in the policy updates [2]. In TD3 the target network's parameters θ are updated with a small rate τ:

$$\theta' \leftarrow \tau\theta + (1 - \tau)\theta' \tag{4}$$

Secondly, target policy smoothing regularization is introduced to reduce the vulnerability to narrow peaks in the value estimate. The main idea is that similar actions in the continuous space should have similar values, which is related to the learning update from State-action-reward-state-action (SARSA) algorithm [11]. To enforce the relationship between similar actions, a small area around the target action is fitted by adding a small amount of Gaussian clipped noise. This results in the following update step:

$$y = r + \gamma Q_{\theta'}(s', \pi_{\phi'}(s') + \epsilon) \quad |\epsilon \sim clip(\mathcal{N}(0, \sigma), -c, c) \tag{5}$$

Moreover, TD3 extends the basic Deep Deterministic Policy Gradient (DDPG) algorithm [6] by resolving the overestimation bias, adding soft target network updates, and smoothing the target policy to further

[2]Implemented and written by Tom Freudenmann

lower the value estimate variance. As a result, it is a state-of-the-art actor-critic algorithm, which shows an improvement in learning speed and performance compared to DDPG in some tasks in continuous domains. We further adapt the proposed implementation to the hockey environment by using different replay buffers (subsection 2.4), reward functions (subsection 2.5) and pink noise (subsection 2.3) as exploration noise.

2.3 Pink Noise[3]

Colored noise as exploration noise for off-policy algorithms was introduced by [1]. It aims to balance out the exploitation-exploitation trade-off. In general, white noise $\mathcal{N}(0, \sigma)$ enforces less exploration and more exploitation, while strong correlated noise like Ornstein-Uhlenbeck focuses on exploring hard actions, resulting in following off-policy trajectories [1]. Colored Noise introduces a new hyperparameter β defining the color of the noise. Therefore, white noise is sampled and transformed with the fast Fourier transform into the frequency domain. The frequencies f are then scaled by $f' = f^{-\beta/2}$ and transformed back into the time domain. We use pink noise ($\beta = 1$), because it is proven to have a positive impact on exploitation in complex tasks [1].

2.4 New Replay Buffers[4]

The basic approach for modeling experience buffers are Experience Replay (ER) buffers [7], which hold the past n (most of the time $n > 100000$) transitions. At training time, m transition batches are sampled to train actor and critic networks. Although each iteration is sampled uniformly, early observations in the empty buffer are drawn more often than late experiences [9]. Instead, we propose first BER, PER [9] and **BPER!** (**BPER!**) [10], to ensure a balanced sampling of late and new experiences.

Balanced Experience Replay BER is a balanced version of the default ER where each experience gets a corresponding memory strength. This strength is reduced every time a new experience is added to the buffer. The resulting probability to draw an experience j from the buffer is $P(j) = \frac{s_j^\alpha}{\sum_i s_i^\alpha}$, where s_i corresponds to the strength of experience i.

Prioritized Experience Replay PER is introduced by [9] and focuses on prioritizing experiences with a big TD-error. Thus, each experience j has a sampling probability $P(j) = \frac{\delta_j^\alpha}{\sum_i \delta_i^\alpha}$, where δ_i is the TD-error of experience i and α is a hyperparameter describing the influence of the priorities. In general, each metric can be used to assign priorities, which may lead to a bias concerning the sampled data. To further balance the sampling, we implement the buffer as Sum Trees, which allow sampling and adding data in $O(log\, n)$ [9].

2.5 Custom Reward Design[5]

The hockey environment already provides a set of reward signals, including a positive signal for winning, the distance to the ball, contact to the ball, and the direction of the pucks' movement. As we observed, our early state agents have problems playing the puck when it starts on their side. Therefore, we added a time penalty, which decreases the overall reward for the complete run, and a penalty for not touching the not moving puck after 10% of the environment steps.

[3] Implemented and written by Tom Freudenmann
[4] Implemented and written by Tom Freudenmann
[5] Implemented and written by Tom Freudenmann

Additionally, TD3 has an issue with aggressive game play. Thus, we added reward for touching the ball when it is moving with negative x-velocity. To further force the agent to minimize its distance to the goal center, we added a linear reward for small distances. To combine all signals, we added weights w_i for all reward signals r_i and provided the agent with the weighted sum over all signals $R = \sum_{i=1}^{8} w_i \cdot r_i$.

2.6 Soft Actor Critic

Our SAC implementation is based on the original paper by [4] and its follow-up paper [5]. SAC is an off-policy actor-critic algorithm that learns a stochastic policy in an environment with continuous action and state spaces. The algorithm is based on the principle of maximum entropy reinforcement learning, which aims to maximize the expected reward while also maximizing the entropy of the policy. This encourages exploration and prevents the policy from getting stuck in local optima. The major components of the SAC algorithm are the actor, the critic, and the entropy target:

$$J(\pi) = \sum_{t=0}^{T} \mathbb{E}_{(s_t,a_t)\sim\rho_\pi} \left[r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot \mid s_t)) \right] \tag{6}$$

The actor is a stochastic policy network, which is based on a vanilla neural network with a Gaussian distribution output layer. It estimates the mean and standard deviation $\mu(s), \sigma(s) = f_\theta(s)$ and provides a sampling method which returns both a sample from the distribution and the log probability of the selected sample. To make the sampling process differentiable, the reparameterization trick is used where $z \sim \mathcal{N}(0, 1)$ and $u = \mu(s) + \sigma(s) \cdot z$. The action bounds are enforced by applying the $\tanh$ function to the sampled unbounded action. To consider this bounding in the probability density function, the probability density function of the unbounded action is scaled by the absolute determinant of the Jacobian matrix of the $\tanh$ function [2][C. Enforcing Action Bounds]:

$$\pi(a \mid s) = \mu(u \mid s) \left| \det \left(\frac{d\mathbf{a}}{d\mathbf{u}} \right) \right|^{-1}. \tag{7}$$

For each state, sampled from the replay buffer $\mathcal{D}$ an action is sampled from the actor and the log probability for this action is calculated using the adjusted probability density function. The actor is trained on the maximum entropy objective, where the critic estimates the value for each state-action pair:

$$J(\pi) = \mathbb{E}_{s_t\sim\mathcal{D}, a_t\sim\pi} \left[\alpha \log \pi(a_t \mid s_t) - Q(s_t, a_t) \right] \tag{8}$$

The critic consists of two Q-networks, valuing state-action pairs. During training both networks independently predict a value for the given state-action pair ($s_t \sim \mathcal{D}, a_t \sim \pi$) and the minimum is selected. This is done to prevent overestimation bias [3].
The target Q-values incorporate the immediate reward r and the discounted future reward. Additionally, an entropy term, weighted by the temperature parameter α, is used to balance exploration and exploitation. This formulation aligns with the objective function $J(\pi)$, which maximizes both the expected reward and the entropy of the policy:

$$y = r + \gamma(1 - d) \left(\min \left(Q_1^{\text{target}}(s', a'), Q_2^{\text{target}}(s', a') \right) + \alpha \mathcal{H}(\pi(\cdot \mid s')) \right). \tag{9}$$

The critic is trained to optimize the mean squared Bellman error, which represents the difference between the estimated Q-values and the target values:

$$\mathcal{L}_Q = \mathbb{E} \left[(Q(s, a) - y)^2 \right]. \tag{10}$$

To stabilize the training process, the target Q-networks are updated using a soft target update mechanism, as it was shown to improve the stability of the training process [4][E. Additional Baseline Results].
In the original paper "an additional function approximator for the value function [was introduced] but later found [as] to be unnecessary" [5]

Moreover the automatically temperature parameter tuning using gradient-based optimization, introduced in the follow-up paper [5], was implemented:

$$J(\alpha) = \mathbb{E}a_t \sim \pi \left[-\alpha \left(\log \pi(a_t \mid s_t) + \mathcal{H}\text{target}\right)\right], \tag{11}$$

where $\mathcal{H}$target is set to negative of the action space dimensionality, as proposed in [5][D. Hyperparameters].

2.7 Hybrid-Model

While reviewing the gameplay of the trained SAC and TD3 agents, we noticed that both agents have their strengths and weaknesses in different situations. To combine the strengths of both agents, we created a hybrid model. This approach is inspired by the Mixture of Experts (MoE) architecture, where multiple experts are combined using a gating network to weight the experts predictions. But since we've got an continious spatial action space, weighting both experts predictions would not be feasible option. Thus a DDQN [13] was implemented as selection gate for sub-agents. This gate estimates the value of choosing a speciffic sub-agent, given a state:

$$i^*(s) = \arg \max_{i \in \text{Sub-Agents}} Q(i, s) \tag{12}$$

$$a = \pi_{i^*(s)}(s) \tag{13}$$

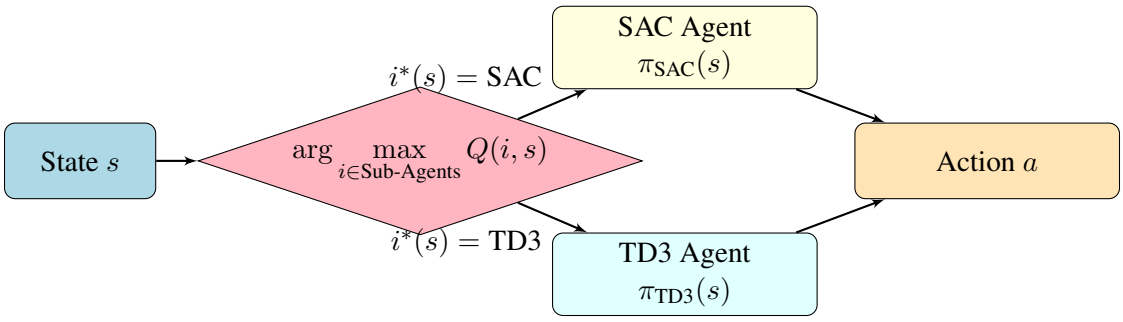

Figure 1: Hybrid model with DDQN-based agent gate using SAC and TD3 as sub-agents.

The loss function for training the DDQN is based on the TD-Error, which is the difference between the predicted and the target Q-value. We investigated three different training strategies for the hybrid model:

- Training from scratch: Training the subagents and the gating network simultaneously from scratch.
- Training by fine-tuning the sub-agents: Using sub-agents that are pre-trained independently and further finetuned simulataneously to the gating network.
- Training with fixed sub agents: Using subagents that are pre-trained independently and left fixed while training the gating network.

3 Evaluation

With all our methodlogies in place, we now investigate their performance in different domains. We start with analysing the sampling behavior of the four implemented buffer types (Section **??**), then we compare the performance on 3 simple environments (Section 3.1), to show the impact of our modifications at well-known reinforcement learning benchmarks. Finally, we are focusing on the hockey environment, were the trainging the foundation models is anlysed and compared to the results of the simple environments (Section 3.2).

3.1 Training on Simple Environments

We trained the introduced algorithms first on simple environments, also used in the lecture. Therefore, we compare TD3 with the different buffers and basic SAC on Pendulum, Lunar Lander and HalfCheetah. In this step we exclude our hybrid approach because it was specifically developed for the hockey environment. With the attached hyperparameters (Appendix B.1), we can observe that SAC has higher average returns after 10000 steps than all TD3 implementations and TD3 with BER outperforms the other buffer implementations in HalfCheetah and Lunar Lander (Table 2). Additionally, PER performs always better than ER. With this results, we expect an equal behavior in the hockey environment.

Table 1: Average Reward for Simple Environments and Algorithm (Last 100 Steps)

Environment	SAC (ER)	TD3 (ER)	TD3 (PER)	TD3 (BER)
HalfCheetah	**3296.86**	1739.69	1874.47	1999.50
Lunar Lander	**143.82**	-41.53	-22.13	61.10
Pendulum	-249.59	-163.91	**-145.48**	-181.35

3.2 Hockey Environment

The performance on the hockey environment is evaluated with the average final performance (Table 2) and the corresponding evaluation reward of the training in 600000 environment steps (Figure 2). Therefore, we train SAC and TD3 on the original environment reward and our own reward. The opponents are the strong/weak basic opponents, SAC foundation, TD3 foundation and Hybrid foundation agents, which are pretrained on the strong/weak opponents. To stabalize training, the opponents change after 10000 environment steps (100 epochs). The evaluation reward contains only the win/lose signal with $r_{eval} \in \{-10, 0, 10\}$ and is recorded every 5000 steps (50 epochs) over 20 runs. The average reward is then saved and plotted in Figure 2. All other statistics, like critic-, actor-loss and training reward provided in the appendix subsection B.5. The final average evaluation reward over 100 runs is also provided in Table 2.

Table 2: Average Evaluation Reward for Hockey Environment

Environment	SAC	TD3	TD3 (White)	TD3 (Blue)	TD3 (PER)	TD3 (BER)
Hockey Original Reward	**8.95**	7.16	-	-	-	-
Hockey Own Reward	9.70	**9.71**	8.56	9.36	0.68	8.22

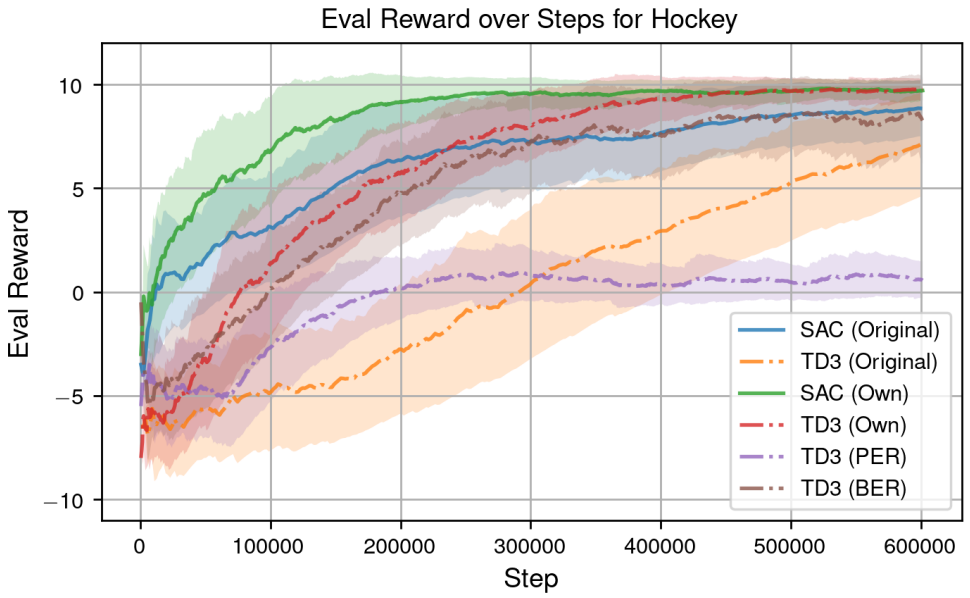

Figure 2: Evaluation Rewards for SAC and TD3 with original and own reward. TD3 is also presented with BER and PER over 600000 environment steps.

Table 3: Performance between the agents and the baseline opponents.

Agent / Opponent	Win-Rate (%) moe-selfplay-expert-training	sac-default-reward	sac-er	td3-er	td3-pink
strong	79.90	97.40	97.70	97.80	98.90
weak	91.00	98.90	99.00	98.80	99.80

References

[1] O. Eberhard, J. Hollenstein, C. Pinneri, and G. Martius. Pink noise is all you need: Colored noise exploration in deep reinforcement learning. In *The Eleventh International Conference on Learning Representations*, 2023.

[2] S. Fujimoto, H. van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. In J. Dy and A. Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1587–1596, Stockholmsmässan, Stockholm Sweden, 10–15 Jul 2018. PMLR.

[3] S. Fujimoto, H. van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods, 2018.

[4] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In J. Dy and A. Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1861–1870, Stockholmsmässan, Stockholm Sweden, 10–15 Jul 2018. PMLR.

[5] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine. Soft actor-critic algorithms and applications. 2019.

[6] T. Lillicrap. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

[7] L.-J. Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine learning*, 8:293–321, 1992.

[8] R. L. Plackett. The analysis of permutations. *Journal of the Royal Statistical Society. Series C (Applied Statistics)*, 24(2):193–202, 1975.

[9] T. Schaul. Prioritized experience replay. *arXiv preprint arXiv:1511.05952*, 2015.

[10] H. Sun, R. Li, H. Yang, and W. Zhu. Balanced prioritized experience replay. In *2022 3rd International Conference on Electronic Communication and Artificial Intelligence (IWECAI)*, pages 200–203, 2022.

[11] R. S. Sutton and A. G. Barto. Reinforcement learning: An introduction. *Robotica*, 17(2):229–235, 1999.

[12] S. Thrun and A. Schwartz. Issues in using function approximation for reinforcement learning. In *Proceedings of the 1993 connectionist models summer school*, pages 255–263. Psychology Press, 2014.

[13] H. van Hasselt, A. Guez, and D. Silver. Deep reinforcement learning with double q-learning, 2015.

[14] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas. Dueling network architectures for deep reinforcement learning. In M. F. Balcan and K. Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1995–2003, New York, New York, USA, 20–22 Jun 2016. PMLR.

A Acronyms

BER Balanced Experience Replay

DDPG Deep Deterministic Policy Gradient

DDQN Double Deep Q-Network

ER Experience Replay

MoE Mixture of Experts

PER Prioritized Experience Replay

SAC Soft Actor Critic

SARSA State-action-reward-state-action

TD Temporal Difference

TD3 Twin Delayed Deep Deterministic Policy Gradient

B Additional Plots and Hyperparameter

B.1 Hyperparameter for evaluation

Table 4: Hyperparameter Summary for Pendulum

Agent	Noise	Actor Sizes	Critic Sizes	Buffer Type	Discount
TD3	σ=0.2, clip=0.8, β=1.0	[128, 128]	[128, 128, 64]	ER	0.95
TD3	σ=0.2, clip=0.8, β=1.0	[128, 128]	[128, 128, 64]	BER	0.95
TD3	σ=0.2, clip=0.8, β=1.0	[128, 128]	[128, 128, 64]	PER	0.95
SAC	N/A	[512, 256]	[512, 256, 64]	ER	0.98

Table 5: Hyperparameter Summary for LunarLander

Agent	Noise	Actor Sizes	Critic Sizes	Buffer Type	Discount
TD3	σ=0.2, clip=0.8, β=1.0	[256, 256]	[256, 256, 128]	ER	0.98
TD3	σ=0.2, clip=0.8, β=1.0	[256, 256]	[256, 256, 128]	BER	0.98
TD3	σ=0.2, clip=0.8, β=1.0	[256, 256]	[256, 256, 128]	PER	0.98
SAC	N/A	[512, 256]	[512, 256, 64]	ER	0.98

Table 6: Hyperparameter Summary for HalfCheetah

Agent	Noise	Actor Sizes	Critic Sizes	Buffer Type	Discount
TD3	σ=0.2, clip=0.8, β=1.0	[256, 256]	[256, 256, 128]	ER	0.95
TD3	σ=0.2, clip=0.8, β=1.0	[256, 256]	[256, 256, 128]	BER	0.95
TD3	σ=0.2, clip=0.8, β=1.0	[256, 256]	[256, 256, 128]	PER	0.95
SAC	N/A	[512, 256]	[512, 256, 64]	ER	0.98

B.2 Evaluation Plots for Pendulum

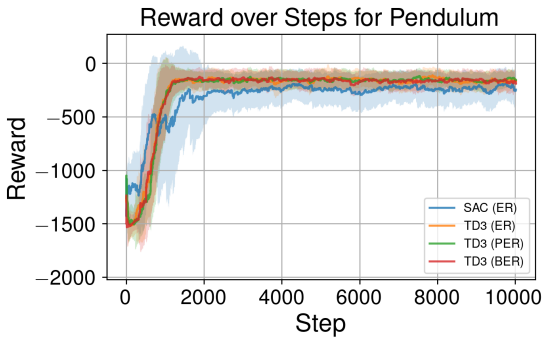

Figure 3: Reward over the environment steps

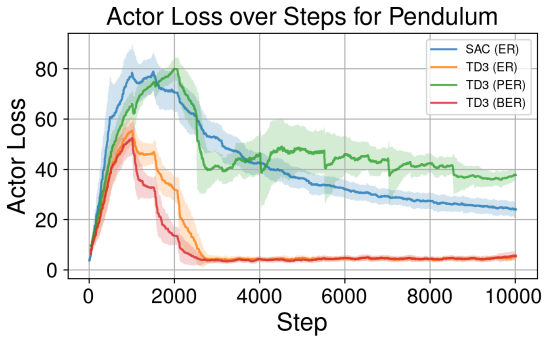

Figure 4: Actor Loss over the environment steps

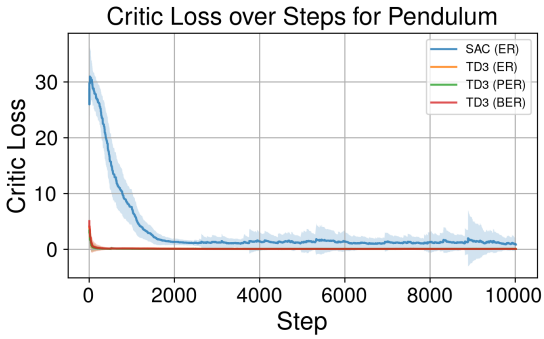

Figure 5: Critic Loss over the environment steps

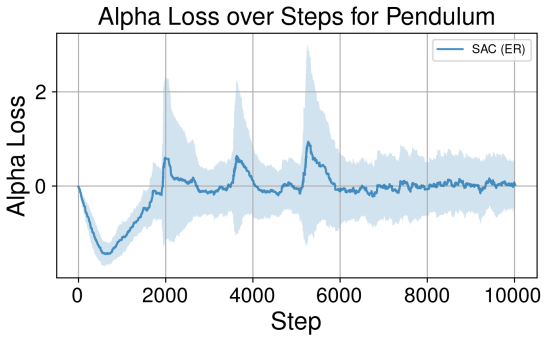

Figure 6: Alpha Loss over the environment steps

B.3 Evaluation Plots for Lunar Lander

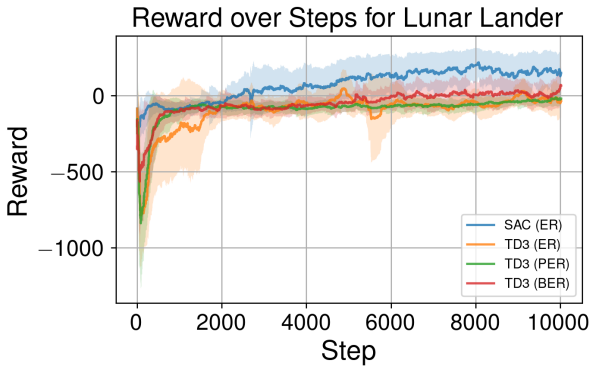

Figure 7: Reward over the environment steps

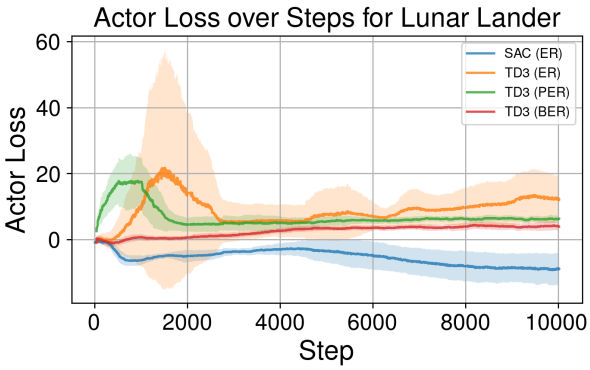

Figure 8: Actor Loss over the environment steps

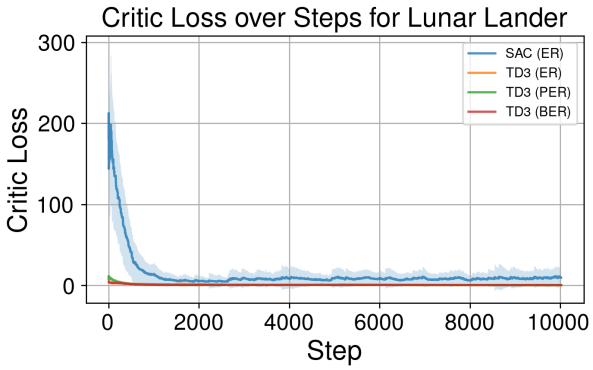

Figure 9: Critic Loss over the environment steps

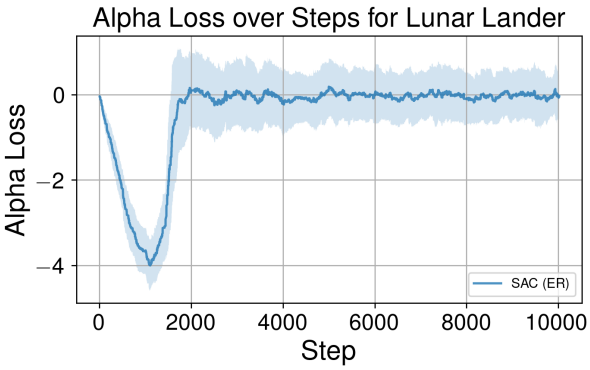

Figure 10: Alpha Loss over the environment steps

B.4 Evaluation Plots for HalfCheetah

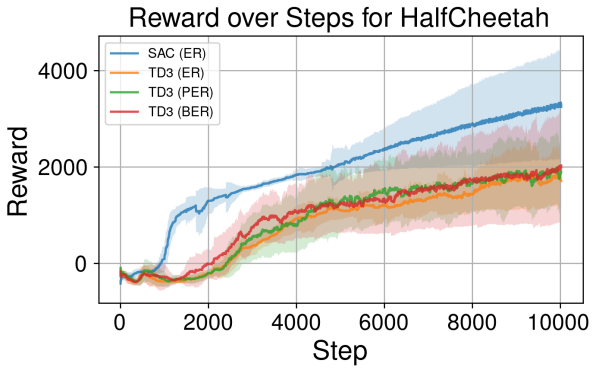

Figure 11: Reward over the environment steps

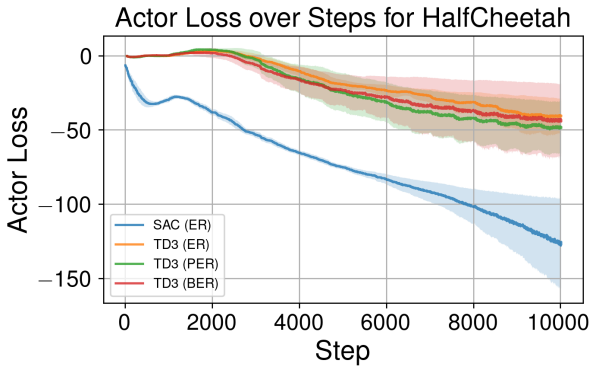

Figure 12: Actor Loss over the environment steps

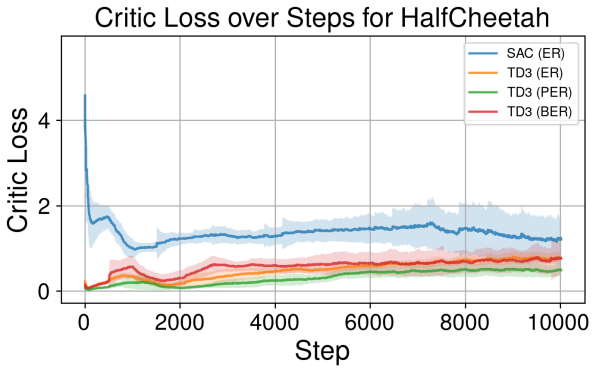

Figure 13: Critic Loss over the environment steps

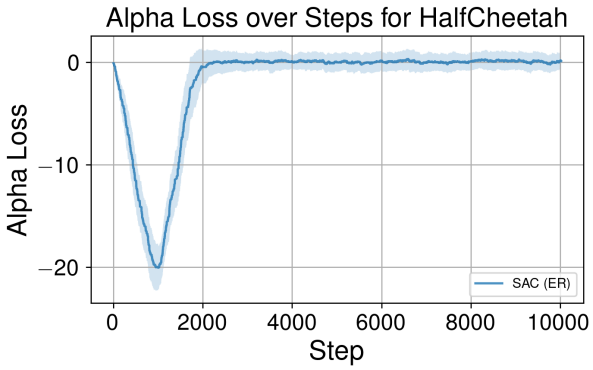

Figure 14: Alpha Loss over the environment steps

B.5 Hockey Evaluation Plots

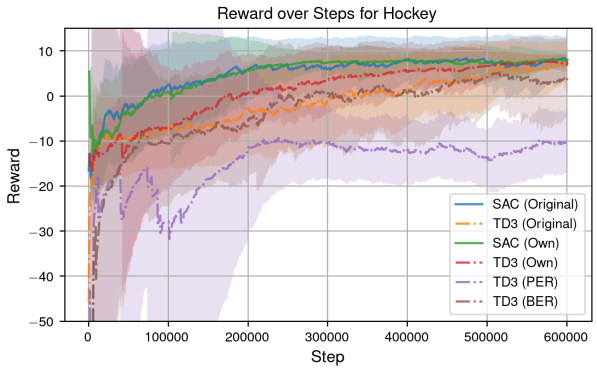

Figure 15: Reward over the environment steps

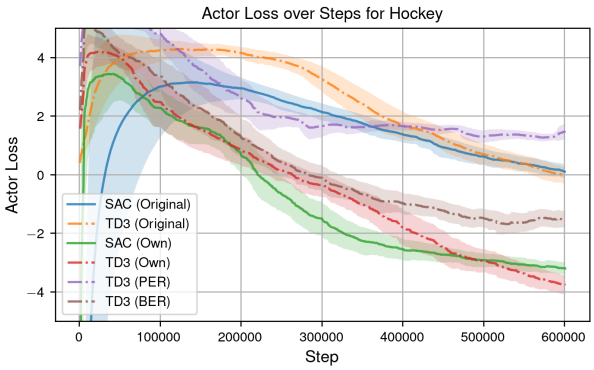

Figure 16: Actor Loss over the environment steps

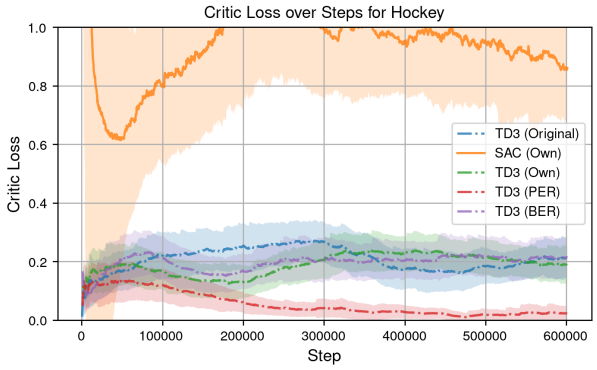

Figure 17: Critic Loss over the environment steps

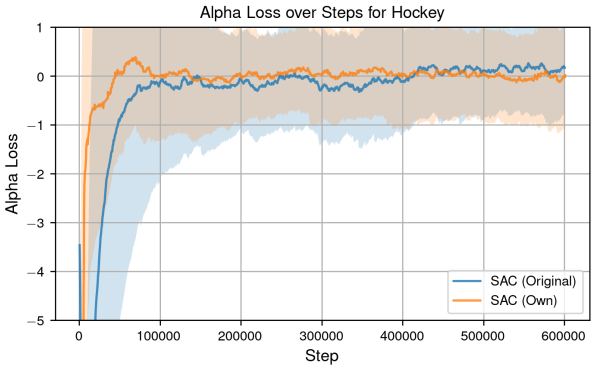

Figure 18: Alpha Loss over the environment steps